



Università della Calabria

Dipartimento di Ingegneria Informatica, Modellistica, Elettronica e Sistemistica
(DIMES)

Primo Progetto di Elettronica Digitale

Sommatore a 16-bit

Francesco Giacobbe e Jacopo Maria Priolo

Matricole 250469 e 204463

A.A. 2025/2026

Indice

0	Introduzione	3
1	Descrizione funzionale del Carry Look-Ahead a 4-bit	3
2	Descrizione funzionale del sommatore a 16-bit	4
2.1	Variante di descrizione architecture: il <i>for-generate</i>	6
3	Simulazione logica del Sommatore a 16-bit	7
3.1	Scelta degli ingressi di benchmark	9
4	Diagramma di timing e Risultati conclusivi	9

0 Introduzione

Nel dominio dei sistemi digitali, le operazioni aritmetiche rappresentano una delle basi fondamentali su cui si fondano i microprocessori. Nello specifico, qualsiasi operazione aritmetica può essere ricondotta a un'addizione binaria. Si evince pertanto che la progettazione e realizzazione di circuiti sommatore efficienti sia di estrema rilevanza. La presente relazione ha come obiettivo l'esposizione dei procedimenti per la realizzazione di un circuito sommatore a 16-bit implementato mediante l'utilizzo di 4 circuiti Carry Look-Ahead a 4 bit.

1 Descrizione funzionale del Carry Look-Ahead a 4-bit

Un circuito sommatore di tipo Carry Look-Ahead è in grado di eseguire la somma di due operandi calcolando in modo parallelo i bit *propagate* e *generate*.

$$\begin{aligned} p_i &= A_i \text{ xor } B_i \\ g_i &= A_i \text{ and } B_i \end{aligned} \quad (1)$$

Ciò rende questo sommatore molto più rapido nel funzionamento rispetto, ad esempio, a un sommatore di tipo Ripple-Carry dove, come suggerisce il nome, ogni sommatore elementare (Full-Adder) deve attendere il calcolo del riporto proveniente dal Full-Adder che lo precede prima di poter determinare la propria uscita.

In particolare, noti i bit propagate e generate (calcolati parallelamente), si sfruttano le seguenti relazioni che li legano ai riporti e ai bit di somma

$$\begin{aligned} c_{i+1} &= g_i + p_i \cdot c_i \\ S_i &= p_i \text{ xor } c_i \end{aligned} \quad (2)$$

per calcolare questi ultimi. Si *srotola* quindi la ricorsione per il calcolo del riporto:

$$\begin{aligned} c_1 &= g_0 + p_0 \cdot c_0 \quad \text{caso base} \\ c_2 &= g_1 + p_1 \cdot c_1 \\ &= g_1 + p_1(g_0 + p_0 c_0) \\ &= g_1 + p_1 g_0 + p_1 p_0 c_0 \\ &\dots \\ c_{i+1} &= g_i + g_{i-1} p_i + g_{i-2} p_{i-1} + \dots + g_0 p_1 p_2 \dots p_i + p_0 p_1 \dots p_i c_0. \end{aligned} \quad (3)$$

Tuttavia, l'aumento di velocità si traduce in un maggiore costo dei componenti. Infatti, come osservabile dalla (3), negli ultimi riporti calcolati, saranno necessarie porte AND e OR fino a $n + 1$ ingressi. Ciò non solo comporta un incremento dei costi di realizzazione, ma richiede anche un'attenta valutazione delle limitazioni imposte dal *fan-in*.

Per la descrizione funzionale di questo circuito sommatore ci spostiamo su ambiente Vivado per descriverlo in linguaggio VHDL. In un file VHDL si distinguono due sezioni:

1. la sezione **entity** che descrive le porte che caratterizzano il circuito, ovvero come si *interfaccia* con l'esterno;
2. la sezione **architecture** che descrive il comportamento interno e la struttura del componente definito nella sezione entity.

```
-- CLA4b.vhd
entity CLA4b is
Port ( A : in bit_vector (3 downto 0);
      B : in bit_vector (3 downto 0);
      cin : in bit;
      cout : out bit;
      S : out bit_vector (3 downto 0));
end CLA4b;

architecture Behavioral of CLA4b is
```

```

-- Segnali interni di riporto, generazione e propagazione.
signal p,g : bit_vector (3 downto 0);
signal c : bit_vector (4 downto 0);

begin
    c(0) <= cin;

    -- Calcolo parallelo dei bit propagate e generate
    p <= A xor B;
    g <= A and B;

    c(1) <= g(0) or (p(0) and c(0));
    c(2) <= g(1) or (p(1) and g(0)) or (p(1) and p(0) and c(0));
    c(3) <= g(2) or (p(2) and g(1)) or (p(2) and p(1) and g(0)) or (p(2) and p(1) and p(0)
        and c(0));
    c(4) <= g(3) or (p(3) and g(2)) or (p(3) and p(2) and g(1)) or (p(3) and p(2) and p(1)
        and g(0)) or (p(3) and p(2) and p(1) and p(0) and c(0));

    cout <= c(4);
    S <= p xor c(3 downto 0);
end Behavioral;

```

A partire da questo codice, inoltre, Vivado ci permette di avere una schematizzazione di questo circuito, osservabile in figura (1).

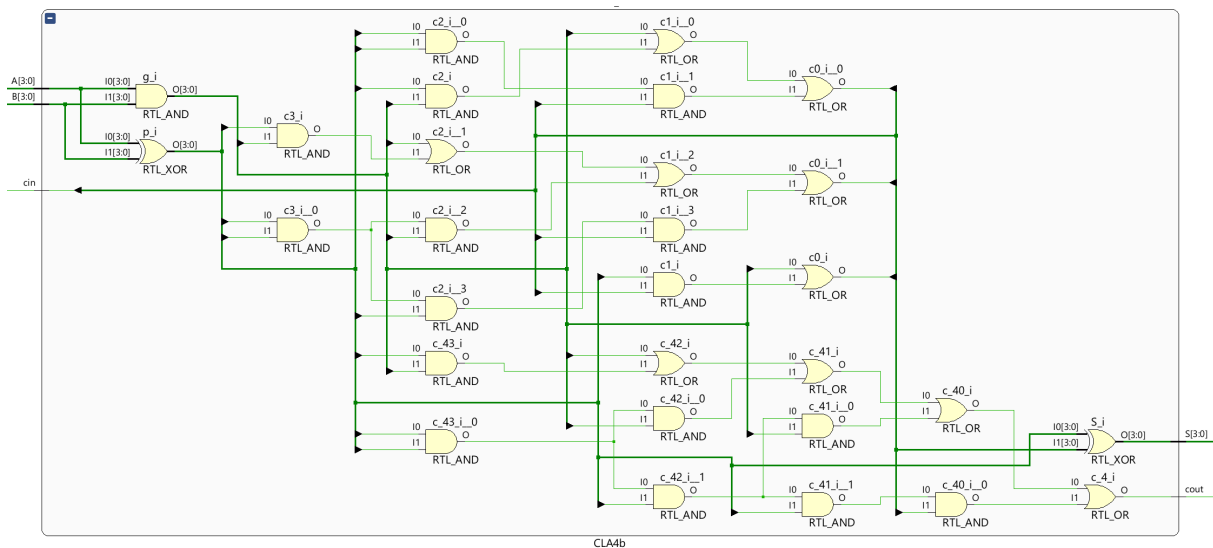


Figura 1: Schema circuitale del Carry Look-Ahead a 4-bit

2 Descrizione funzionale del sommatore a 16-bit

Il sommatore a 16-bit ideato è composto da 4 moduli Carry Look-Ahead a 4-bit la cui descrizione è stata affrontata nel paragrafo precedente. Nello specifico ogni modulo si occupa di sommare un gruppo di 4 bit e di fornire il riporto in uscita al modulo successivo. Sebbene il riporto venga propagato da un modulo all'altro (in maniera analoga a quanto avviene con un Ripple-Carry), all'interno del singolo modulo, trattandosi di Carry Look-Ahead, i riporti vengono calcolati in parallelo. Si ha dunque un parallelismo nel calcolo dei riporti a gruppi di 4 bit e risulta dunque essere una soluzione ibrida di funzionamento tra tecnologia Ripple-Carry e Carry Look-Ahead; di conseguenza, anche la velocità del circuito risulta essere un compromesso tra le due tipologie. Di seguito il codice VHDL che lo descrive e in figura 2 il suo schema circuitale

```

-- Ripple4CLA.vhd
entity Ripple4CLA is
    Port ( A : in bit_vector (15 downto 0);
          B : in bit_vector (15 downto 0);
          cin : in bit;
          cout : out bit;
          S : out bit_vector (15 downto 0));
end Ripple4CLA;

-- Architettura di tipo strutturale
architecture Struct of Ripple4CLA is

    -- Richiamiamo il componente Carry Look-Ahead a 4-bit
    component CLA4b
        Port ( A : in bit_vector (3 downto 0);
              B : in bit_vector (3 downto 0);
              cin : in bit;
              cout : out bit;
              S : out bit_vector (3 downto 0));
    end component;

    -- Utilizziamo un vettore di segnali per gestire la propagazione dei riporti da un
    -- componente CLA al successivo
    signal c_intermedi : bit_vector (2 downto 0);

begin

    -- Istanziamo 4 componenti Carry Look-Ahead a 4-bit andando a mappare le loro porte
    CLA_0 : CLA4b port map (A(3 downto 0), B(3 downto 0), cin, c_intermedi(0), S(3 downto
        0));
    CLA_1 : CLA4b port map (A(7 downto 4), B(7 downto 4), c_intermedi(0), c_intermedi(1),
        S(7 downto 4));
    CLA_2 : CLA4b port map (A(11 downto 8), B(11 downto 8), c_intermedi(1), c_intermedi(2),
        S(11 downto 8));
    CLA_3 : CLA4b port map (A(15 downto 12), B(15 downto 12), c_intermedi(2), cout, S(15
        downto 12));

end Struct;

```

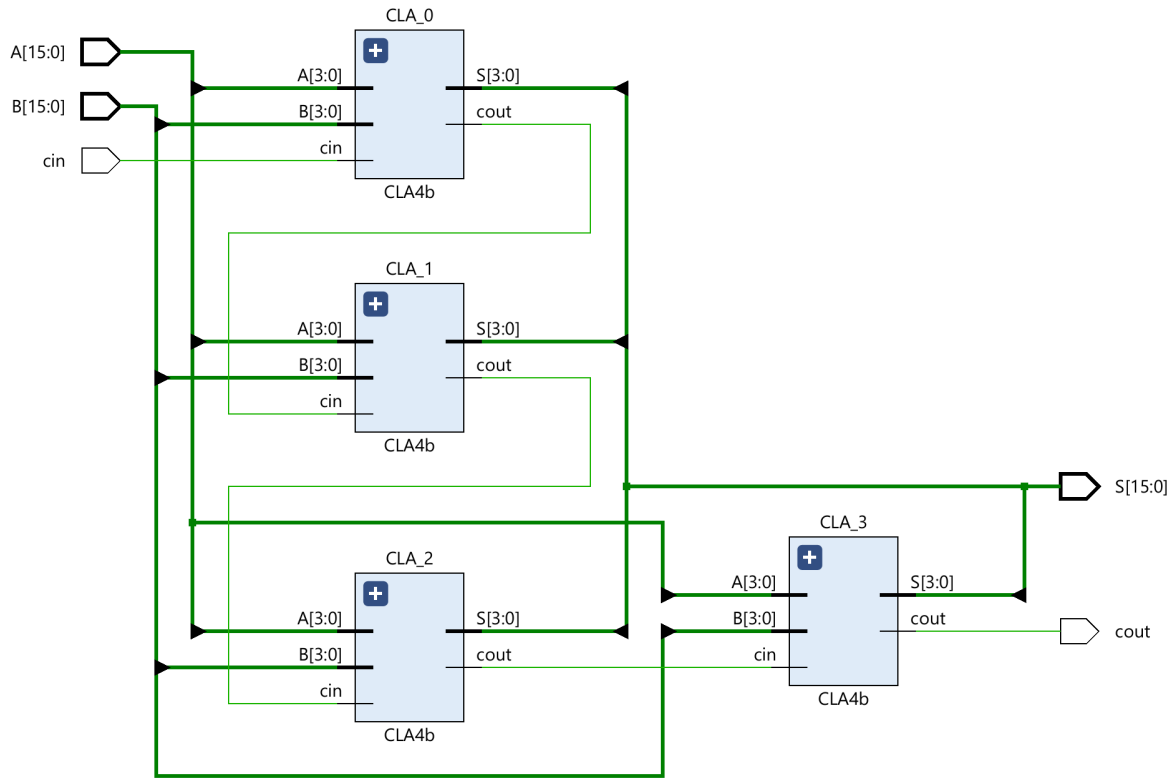


Figura 2: Schema circuitale del sommatore a 16-bit

Si osservi che il corpo della sezione *architecture* in questo caso risulta differente rispetto a quanto avviene per il circuito Carry Look-Ahead a 4-bit. Ciò perché nel caso precedente è stato scelto di descrivere il funzionamento del circuito in maniera *behavioral*, ovvero tramite istruzioni di assegnamento; invece, nel caso del sommatore a 16-bit è stata scelta una descrizione di tipo *structural* (di istanziazione). Il fattore che ha portato a questa scelta di architettura è il semplice fatto che, per sua natura, il sommatore descritto in questa relazione assume una struttura **gerarchica**: si prendono 4 componenti di un livello inferiore (i sommatore Carry Look-Ahead a 4-bit) e li si uniscono per formare un circuito di livello superiore (il sommatore a 16-bit). L'architettura *structural* risulta essere perfetta in questi casi in quanto, per descrivere del circuito, ci abilita a farlo semplicemente istanziando i componenti di livello inferiore e andando a *mapparne* le porte con i segnali di ingresso e uscita del circuito di livello più alto e con eventuali segnali interni ausiliari (nel nostro esempio un segnale ausiliario è il segnale *c_intermedi*). La scelta di architettura *structural* è meglio giustificata osservando la traduzione in schema circuitale in figura (2), dove è anche giustificata la scelta di utilizzo di 3 segnali di riporto intermedi per effettuare un collegamento tra il riporto in uscita di un circuito Carry Look-Ahead e il successivo.

2.1 Variante di descrizione *architecture*: il *for-generate*

Un'alternativa per istanziare all'interno del corpo dell'*architecture* i sottomoduli di tipo Carry Look-Ahead è quella di fare uso di un *for-generate* in modo da poter rendere più pulita la scrittura e per permettere, soprattutto, di generalizzare la descrizione del circuito in modo che lo stesso codice possa risultare valido al variare del numero di bit supportati. Di seguito la variante dove viene utilizzato questo statement, di cui riportiamo solamente ciò che varia rispetto allo snippet di codice precedente:

```
-- Ripple4CLAv2.vhd
...
begin
  c(0) <= cin;
  cout <= c(4);

  -- Istanziamo 4 componenti Carry Look-Ahead a 4-bit andando a mappare le loro porte
  -- utilizzando il FOR GENERATE
  SUBMODULES : for i in 0 to 3 generate
```

```

        CLA_i : CLA4b port map (A(4*i + 3 downto 4*i), B(4*i + 3 downto 4*i), c(i), c(i+1),
                                S(4*i + 3 downto 4*i));
    end generate;
end Struct;
...

```

3 Simulazione logica del Sommatore a 16-bit

La simulazione Logica è stata condotta tramite la realizzazione di un testbench VHDL. L'obiettivo della simulazione è verificare la correttezza funzionale del circuito, ovvero accertarsi che per ogni combinazione di ingressi il circuito fornisca il risultato atteso in termini di somma e riporto finale. È molto importante, nella stesura di un file testbench, dichiarare come *signal* tutte le porte di input e di output del component under testing, oltre a ricordarsi di istanziare quest'ultimo facendone la port map. Una caratteristica peculiare del linguaggio VHDL è che il corpo della sezione architecture contiene, di norma, statement *concorrenti* che vengono eseguiti in parallelo (l'ordine con cui vengono scritti non influisce sul risultato); tuttavia, nel caso della simulazione l'obiettivo è che i segnali di ingresso varino nel tempo. È possibile ottenere questo effetto facendo variare gli ingressi all'interno di un blocco *process*, caratterizzato dall'eseguire le istruzioni in maniera sequenziale piuttosto che concorrente. Ciò ci abilita quindi a far variare gli ingressi nel tempo. Di seguito il codice del file testbench che esegue quanto descritto verbalmente.

```

-- TestRipple4CLA.vhd
entity TestRipple4CLA is
end TestRipple4CLA;

architecture sim of TestRipple4CLA is

    component Ripple4CLA
        Port ( A : in bit_vector (15 downto 0);
              B : in bit_vector (15 downto 0);
              cin : in bit;
              cout : out bit;
              S : out bit_vector (15 downto 0));
    end component;

    signal A : bit_vector (15 downto 0);
    signal B : bit_vector (15 downto 0);
    signal cin : bit;
    signal cout : bit;
    signal S : bit_vector (15 downto 0);

begin
    process begin
        cin <= '0';

        -- Test di riporto in uscita
        A <= (others => '0');
        A(0) <= '1';
        B <= (others => '1');
        wait for 10ns;

        -- Somma di zeri
        A <= (others => '0');
        B <= (others => '0');
        wait for 10ns;

        -- Somma di entrambi gli addendi pari al massimo valore unsigned
        A <= (others => '1');
        B <= (others => '1');
        wait for 10ns;
    end process;
end architecture;

```

```

-- Altri test con valori casuali
A <= "0000000001111100";
B <= "0000000000001000";
wait for 10ns;

A <= "1011010101010100";
B <= "1110001100110011";
wait for 10ns;

A <= "0000011111000001";
B <= "0000011111000001";
wait for 10ns;

A <= "0010101010111100";
B <= "1111100110101001";
wait for 30ns;

-- Ripetiamo gli stessi test, ma con il riporto in entrata pari a 1
cin <= '1';

-- Test di riporto in uscita
A <= (others => '0');
A(0) <= '1';
B <= (others => '1');
wait for 10ns;

-- Somma di zeri
A <= (others => '0');
B <= (others => '0');
wait for 10ns;

-- Somma di entrambi gli addendi pari al massimo valore unsigned
A <= (others => '1');
B <= (others => '1');
wait for 10ns;

-- Altri test con valori casuali
A <= "0000000001111100";
B <= "0000000000001000";
wait for 10ns;

A <= "1011010101010100";
B <= "1110001100110011";
wait for 10ns;

A <= "0000011111000001";
B <= "0000011111000001";
wait for 10ns;

A <= "0010101010111100";
B <= "1111100110101001";
wait for 30ns;
end process;

CUT : Ripple4CLA port map (A, B, cin, cout, S);
end sim;

```

3.1 Scelta degli ingressi di benchmark

Per attestare il corretto funzionamento del circuito sono state definite 14 combinazioni che coprono sia i principali casi limite (test del riporto in uscita, somma di zeri, somma massima) che alcuni ingressi casuali. Si nota alla visione del file di testbench VHDL che i primi 7 ingressi sono relativi a un riporto in ingresso c_{in} pari a 0, mentre i restanti prevedono un riporto in ingresso pari a 1.

Nel nostro caso ciò che abbiamo fatto è una *simulazione selettiva*, dove scegliamo soltanto ingressi limite. Qualora avessimo avuto a che fare con un circuito con un numero ristretto di combinazioni di ingressi avremmo potuto, in alternativa, svolgere una *simulazione esaustiva* dove, come suggerisce il nome, vengono passati in input tutte le possibili combinazioni degli ingressi. Nel caso di un sommatore a 16-bit, naturalmente, è poco pratico svolgere tale tipo di simulazione in quanto il numero di tutte le possibili combinazioni input (considerando anche il riporto in ingresso) sono ben 2^{33} . In ogni caso, in linguaggio VHDL sarebbe stato molto semplice ottenere ciò tramite degli statement *for-loop* innestati all'interno del process.

4 Diagramma di timing e Risultati conclusivi

Durante la simulazione, nella finestra *Waveform* di Vivado si evince la corretta evoluzione temporale dei segnali: ad ogni variazione degli ingressi, le uscite si aggiornano coerentemente a intervalli definiti nel blocco process.

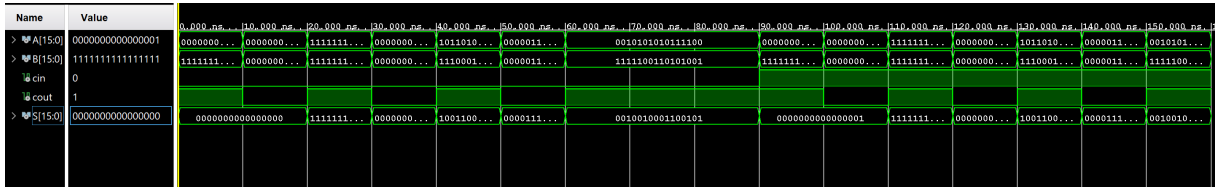


Figura 3: Diagramma di timing

L'analisi del diagramma di timing in figura (3) conferma che:

- la somma binaria è corretta per tutte le combinazioni testate (corretto funzionamento logico);
- la propagazione del riporto tra moduli Carry Look-Ahead avviene correttamente (funzionamento ripple tra CLA).